

```

/**
 * Test for the KalmanFilter class with 1D projectile motion.
 *
 * @author: Hayk Martirosyan
 * @date: 2014.11.15
 */

#include <vector>
#include <Eigen/Dense>

#include "kalman.hpp"

/* Defined in gprof_support.c – zeros arc_table so _mcount works correctly.
 * arc_table is placed by the linker at _end (first byte past BSS), so the
 * CRT's BSS zero-init loop misses it. We zero it explicitly here. */
extern "C" void __monstartup(void);

int main(int argc, char* argv[]) {
    __monstartup(); /* zero arc_table before any _mcount inserts */

    int n = 3; // Number of states
    int m = 1; // Number of measurements

    double dt = 1.0/30; // Time step

    Eigen::MatrixX<double> A(n, n); // System dynamics matrix
    Eigen::MatrixX<double> C(m, n); // Output matrix
    Eigen::MatrixX<double> Q(n, n); // Process noise covariance
    Eigen::MatrixX<double> R(m, m); // Measurement noise covariance
    Eigen::MatrixX<double> P(n, n); // Estimate error covariance

    // Discrete LTI projectile motion, measuring position only
    A << 1, dt, 0, 0, 1, dt, 0, 0, 1;
    C << 1, 0, 0;

    // Reasonable covariance matrices
    Q << .05, .05, .0, .05, .05, .0, .0, .0, .0;
    R << 5;
    P << .1, .1, .1, .1, 10000, 10, .1, 10, 100;

    // Construct the filter
    KalmanFilter kf(dt, A, C, Q, R, P);

    // List of noisy position measurements (y)
    std::vector<double> measurements = {
        1.04202710058, 1.10726790452, 1.2913511148, 1.48485250951, 1.72825901034,
        1.74216489744, 2.11672039768, 2.14529225112, 2.16029641405, 2.21269371128,
        2.57709350237, 2.6682215744, 2.51641839428, 2.76034056782, 2.88131780617,
        2.88373786518, 2.9448468727, 2.82866600131, 3.0006601946, 3.12920591669,
        2.858361783, 2.83808170354, 2.68975330958, 2.66533185589, 2.81613499531,
        2.81003612051, 2.88321849354, 2.69789264832, 2.4342229249, 2.23464791825,
        2.30278776224, 2.02069770395, 1.94393985809, 1.82498398739, 1.52526230354,
        1.86967808173, 1.18073207847, 1.10729605087, 0.916168349913, 0.678547664519,
        0.562381751596, 0.355468474885, -0.155607486619, -0.287198661013, -0.602973173813
    };

    // Best guess of initial states
    Eigen::VectorX<double> x0(n);
    double t = 0;
    x0 << measurements[0], 0, -9.81;
    kf.init(t, x0);

    // Feed measurements into filter, output estimated states

    Eigen::VectorX<double> y(m);
    for(int i = 0; i < (int)measurements.size(); i++) {
        t += dt;

```

```
    y << measurements[i];  
    kf.update(y);  
}  
  
return 0;  
}
```